

Question 1: Your name and ID.

Andrew Branum, 5291638

Question 2: Did you fully implement the project as described? If not, what parts are not implemented at all, or what parts are not implemented as specified in the project description?

Yes

Question 3: For each part, briefly describe how you accomplished the task, and share any details that is asked from you in the project description, such as screenshots, plots, etc., to prove your work.**Part 1:**

To create the clockserver.go program from the given clock2.go program only a few modifications had to take place. First, accepting a port number to listen on via the command line. I chose to use the flag package as it supports creating custom command line flags with specific types and default values. To do this, I created a custom String flag “-p” with the default value of “8080” and assigned the returned value to var portNumber. Since the return value is the address of the string variable that stores the value of the flag, access to the value is obtained with *portNumber. Following the project guidelines, I then dynamically assign the port to an address var prefaced with “:” to create the listening address “:portNumber”. This var is passed into the net.Listen function as the address parameter and the program starts to listen at that address. Other than adding the time zone to the time formatting, the rest of the program stays the same.

The clockwall.go program is also very similar to the given netcat1.go program. For cleanliness, I chose to move the code located in the main goroutine of netcat1 to a function names connectToServer that takes the server address and a pointer to the global sync.WaitGroup as parameters. I start by creating a WaitGroup assigned to wg, then I iterate over all of the command line arguments past index 1. For each argument, I add 1 to the WaitGroup then start a goroutine containing the connectToServer function with the command line argument and address of the WaitGroup being passed in each iteration. Once the loop terminates, the program blocks until all goroutines are terminated.

Setting up the EC2 instances was straightforward, I simply followed the video lecture to configure 2 t2.micro instances with Ubuntu 24.04. I configured them to allow all in/outbound traffic and luckily didn’t have any security problems. One instance is configured with UTC time and the other instance is configured with Australia/Sydney’s time zone. I deployed instance 1 (utc) on 3.146.178.127:8080 and instance 2 (aedt) on

3.145.83.190:8000. When running my clockwall.go from the cmd prompt I am able to connect to both instances and display their respective times as seen below.

```
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1021-aws amd64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/support

System information as of Thu Feb  6 03:10:16 UTC 2025

System load: 0.22      Processes:    107
Usage of /:  42.4% of 4.7GiB    Users logged in: 0
Memory usage: 21%      IPv6 address for enb0: 172.31.9.212
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

17 updates can be applied immediately.
18 of these updates are standard security updates.
To see these additional updates run apt list --upgradable
5 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu Feb  6 14:08:08 2025 from 3.16.146.3
ubuntu@ip-172-31-9-212:~$ cd ~/go/src/clockserver
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ make clockserver.go
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ date
Thu Feb  6 03:10:18 UTC 2025
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ go build -o clockserver clockserver.go
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ sudo dpkg-reconfigure tzdata

Current default time zone: 'Europe/Dublin'
local time is now:      Thu Feb  6 03:10:13 GMT 2025.
Universal time is now:  Thu Feb  6 03:10:13 UTC 2025.

ubuntu@ip-172-31-9-212:~/go/src/clockserver$ date
Thu Feb  6 03:10:15 GMT 2025
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ date
Thu Feb  6 03:10:19 GMT 2025
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ sudo dpkg-reconfigure tzdata

Current default time zone: 'Australia/Sydney'
local time is now:      Thu Feb  6 14:15:43 AEST 2025.
Universal time is now:  Thu Feb  6 03:15:43 UTC 2025.

ubuntu@ip-172-31-9-212:~/go/src/clockserver$ date
Thu Feb  6 14:04:45 AEST 2025
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ ./clockserver -p 8000
2025/02/06 14:17:09 Port Number: 8000
2025/02/06 14:17:09 Host Location: 8000
|

I-0931adaf28457724 (clockServer1)
PublicIP: 146.81.190  PrivateIP: 172.31.9.212
```

```
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1021-aws amd64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/support

System information as of Thu Feb  6 03:10:24 UTC 2025

System load: 0.78      Processes:    107
Usage of /:  40.9% of 4.7GiB    Users logged in: 0
Memory usage: 21%      IPv6 address for enb0: 172.31.6.168
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

14 updates can be applied immediately.
18 of these updates are standard security updates.
To see these additional updates run apt list --upgradable
5 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu Feb  6 14:04:55 2025 from 3.16.146.3
ubuntu@ip-172-31-6-168:~$ cd ~/go/src/clockserver
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ make clockserver.go
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ go build -o clockserver clockserver.go
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ date
Thu Feb  6 03:10:51 UTC 2025
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ date
Thu Feb  6 03:10:57 UTC 2025
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ ./clockserver -p 8080
2025/02/06 03:17:12 Port Number: 8080
2025/02/06 03:17:12 Host Location: 8080
|

I-0931adaf28457724 (clockServer1)
PublicIP: 146.81.190  PrivateIP: 172.31.9.212
```

```
C:\Users\j\OneDrive\Desktop\go\proj1\clockwall>
81:22:49 UTC
81:22:49 AEST
81:22:50 UTC
81:22:51 AEST
81:22:51 UTC
81:22:52 AEST
81:22:52 UTC
81:22:53 AEST
81:22:53 UTC
81:22:54 AEST
81:22:54 UTC
81:22:55 AEST
81:22:55 UTC
81:22:56 AEST
81:22:56 UTC
81:22:57 AEST
81:22:57 UTC
81:22:58 AEST
81:22:58 UTC
81:22:59 AEST
81:22:59 UTC
81:23:00 AEST
81:23:00 UTC
81:23:01 AEST
81:23:01 UTC
81:23:02 AEST
81:23:02 UTC
```

```
I-0931adaf28457724 (clockServer1)
PublicIP: 146.81.190  PrivateIP: 172.31.9.212
```

```
I-0931adaf28457724 (clockServer1)
PublicIP: 146.81.190  PrivateIP: 172.31.9.212
```

```
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ date
Thu Feb  6 14:16:45 AEDT 2025
ubuntu@ip-172-31-9-212:~/go/src/clockserver$ ./clockserver -p 8000
2025/02/06 14:17:09 Port Number: 8000
2025/02/06 14:17:09 Host Location: :8000
|
```

```
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ date
Thu Feb  6 03:15:07 UTC 2025
ubuntu@ip-172-31-6-168:~/go/src/clockserver$ ./clockserver -p 8080
2025/02/06 03:17:12 Port Number: 8080
2025/02/06 03:17:12 Host Location: :8080
|
```

```
C:\Users\Drew\Desktop\G0\proj1\clockwall>go run clockwall.go 3.146.178.127:8080 3.145.83.190:8080
03:22:49 UTC
14:22:49 AEDT
03:22:50 UTC
14:22:50 AEDT
03:22:51 UTC
14:22:51 AEDT
03:22:52 UTC
14:22:52 AEDT
03:22:53 UTC
14:22:53 AEDT
03:22:54 UTC
14:22:54 AEDT
14:22:55 AEDT
03:22:55 UTC
03:22:56 UTC
14:22:56 AEDT
03:22:57 UTC
14:22:57 AEDT
03:22:58 UTC
14:22:58 AEDT
03:22:59 UTC
14:22:59 AEDT
03:23:00 UTC
14:23:00 AEDT
03:23:01 UTC
14:23:01 AEDT
03:23:02 UTC
14:23:02 AEDT
```

Instance summary for i-0933aa6af28457724 (clockServer1) info

Updated 15 minutes ago

Instance ID

i-0933aa6af28457724

IPv6 address

-

Hostname type

IP name: ip-172-31-9-212.us-east-2.compute.internal

Answer private resource DNS name

-

Auto-assigned IP address

3.145.83.190 [Public IP]

IAM Role

-

IMDSv2

Required

Operator

-

Public IPv4 address

3.145.83.190 | [open address](#)

Instance state

Running

Private IP DNS name (IPv4 only)

ip-172-31-9-212.us-east-2.compute.internal

Instance type

t2.micro

VPC ID

vpc-07c291187dadd93bc

Subnet ID

subnet-077c283e1b84d4ecb

Instance ARN

arn:aws:ec2:us-east-2:271747197905:instance/i-0933aa6af28457724

Private IPv4 addresses

172.31.9.212

Public IPv4 DNS

ec2-3-145-83-190.us-east-2.compute.amazonaws.com | [open address](#)

Elastic IP addresses

-

AWS Compute Optimizer finding

[Opt-in to AWS Compute Optimizer for recommendations. | Learn more](#)

Auto Scaling Group name

-

Managed

false

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

▼ Instance details info

AMI ID

ami-0cb91c7de36eed2cb

AMI name

ubuntu/images/hvm-ssd-ubuntu-noble-24.04-amd64-server-20250115

Monitoring

disabled

Allowed image

-

Platform details

Linux/UNIX

Termination protection

Disabled

Instance summary for i-0bf9d6bd2e23698fb (cl...)

Updated 16 minutes ago

Instance ID
i-0bf9d6bd2e23698fb

IPv6 address
-

Hostname type
IP name: ip-172-31-6-168.us-east-2.compute.internal

Answer private resource DNS name
IPv4 (A)
-

Auto-assigned IP address
3.146.178.127 [Public IP]

IAM Role
-

IMDSv2
Required

Operator
-

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-6-168.us-east-2.compute.internal

Instance type
t2.micro

VPC ID
vpc-07c291187d933bc

Subnet ID
subnet-077c283e1b84d4ecb

Instance ARN
arn:aws:ec2:us-east-2:271747197905:instance/i-0bf9d6bd2e23698fb

Private IPv4 addresses
172.31.6.168

Public IPv4 DNS
ec2-3-146-178-127.us-east-2.compute.amazonaws.com | open address

Elastic IP addresses
-

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name
-

Managed
false

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

▼ Instance details Info

AMI ID
ami-0cb91c7de36eed2cb

AMI name
ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-20250115

Stop protection
Disabled

Monitoring
disabled

Allowed image
-

Launch time
Wed Feb 05 2025 22:09:34 GMT-0500 (Eastern Standard Time) (1 minute)

Platform details
Linux/UNIX

Termination protection
Disabled

AMI location
amazon/ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-20250115

Part 2:

A process is a program in execution that includes code, a stack, a data section, and a heap. Each process has its own memory space and can only access its own address space meaning processes are isolated from each other. Kernel threads in a process share the same address space making it easier to context switch. They are also cheaper to create and destroy compared to processes. Kernel threads generally have a fixed size, are directly scheduled by the kernel, and must save its registers and state in memory and update the scheduler when passing control from one thread to another. Goroutines are different such that their stack size grows and shrinks as needed (2KB-1GB) and the Go runtime multiplexes m goroutines on n OS thread (dynamic scheduling). Rescheduling goroutines is much cheaper and more efficient since they utilize user-level switching rather than kernel-level. Their scheduler also doesn't operate periodically but rather implicitly.

Concurrent Programming as defined by the book is "the expression of a program as a composition of several autonomous activities." Concurrency is being able to maintain multiple portions of a program simultaneously. These portions can be executing together asynchronously or synchronously and can purposefully block (wait) until another completes. Parallelism is spreading the portions of the program out to different CPUs to execute at the same time. The key difference is concurrency can occur on a single CPU where parallelism requires multiple CPUs.

Surface_s.go:

After removing the I/O operations and changing the cells to 12000, I ran the program multiple times yielding: 14.370, 14.066, 13.884 and 13.958. I made sure the cache was cleared using `go clean -cache -testcache -modcache`. To ensure both programs performed the same calculation, I lowered the number of cells to 5 and outputted them to the console where I copy and pasted them into a notepad and compared. The data was the same.

```
Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_s
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_s
$ time go run surface_s.go

real    0m13.958s
user    0m0.000s
sys     0m0.031s
```

I then tested this on the t2.2xlarge EC2 instance resulting in ~20 seconds no matter the GOMAXPROC value.

Surface_p.go:

To implement the parallel version of the surface program I used the same method of capturing the command line flag as the first portion of the project. I then assigned this value to the GOMAXPROC via the runtime package. A waitgroup is used to maintain the goroutines. I chose to split the work amongst the goroutines as evenly as possible by dividing the number of cells by the GOMAXPROC value. A for loop then iterates over the number of workers launching a goroutine that calculates the values for its respective cells. After this loop completes and all the goroutines are launched, the waitgroup blocks until all of them are completed and then the application exits.

Since my PC has 8 CPU cores and 32 gigabytes of RAM, I opted to increase the number of cells to 12000 so there is a more distinct change in calculation time for the parallel portion of this project. I started with 2 and ran the program 3 times per increase yielding: 2 (10.212,10.153, 10.180), 4(7.444,7.855,7.677), 6(6.303,6.367,6.379), 8(6.015,5.848,6.085). 10(5.853,5.759,5.852), 12(5.857,5.411,6.170), 14(6.404, 5.790, 5.276), 16(5.243,6.016,5.253).

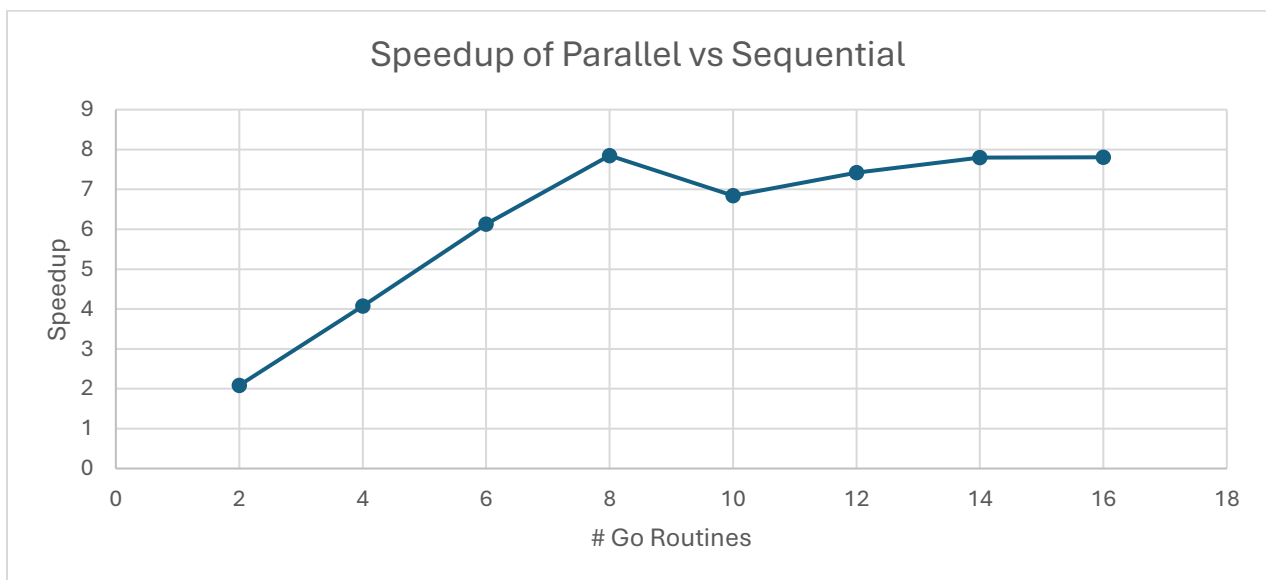
CPUs	Time (Avg)
2	10.182
4	7.659
6	6.349
8	5.983
10	5.821

12	5.813
14	5.823
16	5.504

After running the application 3 times for each GOMAXPROC values, it looks like the optimal value is 8 and anything above that offers minimal to no improvement.

The t2.2xlarge machine has 8 CPU cores as well with 32 gigabytes of memory. After configuring the EC2 instance the following was noted. 2(9.584, 9.587,9.593), 4(4.884, 4.927,4.914), 6(3.272, 3.215, 3.307), 8(2.514, 2.565, 2.565), 10 (3.009, 3.004, 2.754), 12(2.662, 2.655, 2.769), 14(2.585, 2.538, 2.571), 16(2.586, 2.552, 2.548)

CPUs	Time (Avg)	Speedup
2	9.588	2.086
4	4.908	4.075
6	3.265	6.126
8	2.548	7.849
10	2.922	6.845
12	2.695	7.421
14	2.565	7.797
16	2.562	7.806



The calculated speedup for each number of go routines was 20 divided by the average time for each number of go routines. It maxes out at 8 which makes sense considering the VM had 8 vCPUs and anything above that just adds overhead and complexity. 16 go routines

got close but didn't surpass 8 further reinforcing the ideal number of go routines being the amount of CPUs the machine has.

```
$ time go run surface_p.go -g 2

real    0m10.212s
user    0m0.000s
sys     0m0.031s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 2

real    0m10.153s
user    0m0.000s
sys     0m0.030s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 2

real    0m10.180s
user    0m0.000s
sys     0m0.015s
```

```

$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 4

real    0m7.855s
user    0m0.000s
sys     0m0.015s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 4

real    0m7.677s
user    0m0.000s
sys     0m0.031s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 6

real    0m6.303s
user    0m0.015s
sys     0m0.016s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 6

real    0m6.367s
user    0m0.000s
sys     0m0.016s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 6

real    0m6.379s
user    0m0.015s
sys     0m0.030s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 8

real    0m6.015s
user    0m0.000s
sys     0m0.046s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 8

real    0m5.848s
user    0m0.000s
sys     0m0.015s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 8

real    0m6.299s
user    0m0.000s
sys     0m0.046s

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ go clean -cache -testcache -modcache

Drew@DESKTOP-MP3MOLK MINGW64 ~/Desktop/G0/proj1/surface_p
$ time go run surface_p.go -g 8

real    0m6.085s
user    0m0.000s
sys     0m0.031s

```



```

real    0m9.584s
user    0m19.146s
sys     0m0.006s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 2

real    0m9.587s
user    0m19.154s
sys     0m0.008s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 2

real    0m9.593s
user    0m19.144s
sys     0m0.012s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 4

real    0m4.884s
user    0m19.284s
sys     0m0.011s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 4

real    0m4.927s
user    0m19.362s
sys     0m0.017s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 4

real    0m4.914s
user    0m19.348s
sys     0m0.009s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 6

real    0m3.272s
user    0m19.297s
sys     0m0.014s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 6

real    0m3.215s
user    0m19.145s
sys     0m0.006s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 6

real    0m3.307s
user    0m19.334s
sys     0m0.018s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 8

real    0m2.514s
user    0m19.512s
sys     0m0.010s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 8

real    0m2.565s
user    0m19.555s
sys     0m0.015s
ubuntu@ip-172-31-0-87:~/go/src$ sudo sh -c 'sync && echo 3 > /proc/sys/vm/drop_caches'
ubuntu@ip-172-31-0-87:~/go/src$ time ./surface_p -g 8

real    0m2.565s

```

Instance summary for i-04b747ededa9ded94 (2xLarge) info

Updated less than a minute ago

Instance ID

 i-04b747ededa9ded94

IPv6 address

–

Hostname type

IP name: ip-172-31-0-87.us-east-2.compute.internal

Answer private resource DNS name

IPv4 (A)

Auto-assigned IP address

 18.222.115.22 [Public IP]

IAM Role

–

IMDSv2

Required

Operator

–

Public IPv4 address

 18.222.115.22 | [open address](#) 

Instance state

 **Running**

Private IP DNS name (IPv4 only)

 ip-172-31-0-87.us-east-2.compute.internal

Instance type

t2.2xlarge

VPC ID

 vpc-07c291187dacd93bc 

Subnet ID

 subnet-077c283e1b84d4ecb 

Instance ARN

 arn:aws:ec2:us-east-2:271747197905:instance/i-04b747ededa9ded94

Private IPv4 addresses

 172.31.0.87

Public IPv4 DNS

 ec2-18-222-115-22.us-east-2.compute.amazonaws.com | [open address](#) 

Elastic IP addresses

–

AWS Compute Optimizer finding

 [Opt-in to AWS Compute Optimizer for recommendations.](#) | [Learn more](#) 

Auto Scaling Group name

–

Managed

false



Connect

Instance state ▼

Actions ▼